

Venta–Usuario e Historial de Caja (V5)

Todo lo que se hizo en esta tanda: qué clase resuelve qué, cómo se relacionan, por qué se hizo así y el código que se usó. Backend Spring Boot + Frontend Angular.

Campo	Contenido
Fecha	2026-10-01
Alcance	Backend: ventas ligadas a usuarios, fechas de alta/baja de usuarios, re-alta de caja, endpoint de historial de caja con filtros. Frontend: paginación del carrito de compras, flechas de la paginación, modal de cajas, historial de caja en Reportes, filtros de usuarios dados de baja, alineación de columnas.
Base de datos	Migración nueva V5 (no se editó ninguna anterior)
Verificación	mvnw test → 258/258 · ng build → OK

Antes de empezar: dos cosas NO se hicieron a propósito

- 1. No se rellenó el usuario de las ventas viejas.** El dato no existía, así que quedó en NULL. Inventar un usuario sería peor que no tener el dato.
- 2. No se cambió la tabla de ventas a 'una fila por usuario'.** Los totales por usuario se calculan al vuelo en el reporte, que es menos código y no duplica la información.

1. El mapa de clases, en una imagen

```
users
(UserEntity)
-----
id
name
email UNIQUE
role_id --> roles
active
activated_at (V5)
deactivated_at (V5)
|
|
|
+----->
(V5)

sales
(SaleEntity)
-----
+--> id
| sale_date
| total
| payment_method
| cash_register_id --> cash_registers
| user_id --> users <-----+ (V5 NUEVO)
|
| cash_registers
| (CashRegisterEntity)
| -----
| id
| number (copia historica)
| opened_at / closed_at / active
| opening_amount / expected_amount
| difference / difference_reason
| cash_box_id --> cash_boxes (V4)
| cash_sales / debit_sales / ...

cash_boxes
(CashBoxEntity)
-----
id
number UNIQUE
description
active
```

Lo nuevo de esta tanda son las dos relaciones marcadas **V5**: `sales.user_id` y las dos fechas en `users`. Todo lo demás ya existía.

1.1 Quién hace qué

Clase / archivo	Responsabilidad	Qué cambió
V5__ventas_usuario...	Las dos columnas nuevas y su backfill	NUEVA
SaleEntity	La venta, ahora con su usuario	+ user

Clase / archivo	Responsabilidad	Qué cambió
SaleImpl	Lee el usuario del JWT y lo pone en la venta	+ currentUserOrNull
UserEntity	El usuario, ahora con fechas de alta y baja	+ 2 campos
UserImpl	Mantiene coherentes esas fechas en alta/baja	+ lógica
UserDto / UserMapper	Exponen las fechas al frontend	+ 2 campos
CashRegisterService/Impl	Re-alta de una caja dada de baja	+ activateBox
CashBoxReportDTO	El DTO del historial de caja (caja + turnos + vendedores)	NUEVO
ReportsService/Impl	El filtro por usuario y fechas, y el agrupado por vendedor	+ método
SaleMapper	Copia el nombre del usuario al DTO de venta	+ 6 líneas

2. La migración V5

Es una migración **nueva**. Nunca se edita una migración que ya se aplicó: si se edita, Flyway cambia el checksum de la versión que ya está en la base y la app deja de arrancar.

2.1 Qué agrega

```
ALTER TABLE public.sales
    ADD COLUMN IF NOT EXISTS user_id bigint;

CREATE INDEX IF NOT EXISTS idx_sales_user ON public.sales (user_id);

ALTER TABLE public.sales
    ADD CONSTRAINT fk_sales_user
    FOREIGN KEY (user_id) REFERENCES public.users(id)
    ON DELETE SET NULL;      <-- importante, ver abajo

ALTER TABLE public.users ADD COLUMN IF NOT EXISTS activated_at    timestamp(6);
ALTER TABLE public.users ADD COLUMN IF NOT EXISTS deactivated_at timestamp(6);

-- Índice PARCIAL: solo las filas dadas de baja se buscan por fecha
CREATE INDEX IF NOT EXISTS idx_users_deactivated_at
    ON public.users (deactivated_at)
    WHERE active = false;
```

2.2 Las dos decisiones del diseño

ON DELETE SET NULL, y no CASCADE

Si se borra un usuario, sus ventas **no** se borran. Con CASCADE, borrar un usuario borraría el historial de ventas del local, que es un dato del negocio y no del empleado. Con SET NULL la venta sobrevive y queda sin dueño, que es la verdad: nadie puede decir quién la hizo. Perder una venta es peor que perder un dato de ella.

Por qué NULL y no un usuario inventado

Las ventas anteriores a V5 no tienen dueño porque el dato nunca se guardó. Ponerle el admin, o el primer usuario, sería fabricar información: el reporte mostraría 'este usuario vendió 500' y sería falso. Un NULL se muestra como 'Sin usuario' y no miente.

El índice de deactivated_at es **parcial** (tiene WHERE active = false) porque la única consulta que lo usa es la de la tabla de datos de baja. Indexar toda la tabla para eso sería gastar espacio en las filas activas, que son las que nunca se buscan por esa columna.

3. La venta ligada a su usuario

3.1 La entidad

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "user_id")
private UserEntity user;
```

LAZY porque el filtro por usuario solo necesita el id (que ya viene en la propia venta) y el nombre se pide en la consulta de reportes. Con EAGER, listar el historial de ventas haría un SELECT extra por venta: es el problema clásico de N+1.

3.2 De dónde sale el usuario

Del **contexto de seguridad**, no del body de la petición. Esta es la decisión de seguridad más importante de esta tanda:

```
private UserEntity currentUserOrNull(){
    Authentication auth = SecurityContextHolder.getContext().getAuthentication();

    if(auth == null || !auth.isAuthenticated()){
        return null;
    }
    // "anonymousUser" es lo que Spring pone cuando no hay token: no es un
    // usuario real y no debe buscarlo en la base.
    if(!(auth.getPrincipal() instanceof UserDetails details)){
        return null;
    }
    return userRepository.findByEmail(details.getUsername()).orElse(null);
}
```

Por qué NO pedir el usuario en el JSON

Si el endpoint aceptara {"userId": 7} en el cuerpo, cualquiera con un token válido podría registrar ventas a nombre de otro. El reporte de 'quién vendió' dejaría de ser confiable, que es justo lo que se agregó para poder auditar. El usuario siempre sale del token, que ya fue validado.

Que devuelva **null** en vez de fallar es deliberado: los tests de integración no levantan el filtro de seguridad, y una llamada interna no lo tiene. El costo es una venta sin usuario, que es exactamente el mismo estado que tienen las ventas anteriores a V5.

4. Las fechas de alta y baja de los usuarios

`users.active` es un booleano: dice **sí** está dado de baja, pero no **cuándo**. Con eso la pregunta 'buscar el registro de la persona que se fue en abril' no tiene respuesta.

```
createUser    -> setActivatedAt(now); setDeactivatedAt(null);
activateUser  -> setActivatedAt(now); setDeactivatedAt(null);    // re-activar
deactivateUser -> if(estaba activo) setDeactivatedAt(now);
                  setActivatedAt(null);                          // ver abajo
```

4.1 Por qué la baja no pisa la fecha si ya estaba dado de baja

La guarda `if(user.isActive())` parece redundante y no lo es. Si se llama dos veces *dar de baja*, la segunda actualizaría la fecha y se perdería la **original**. Y la original es la que responde '¿desde cuándo se fue?'. La fecha de baja es un hecho, no un estado que se pueda reescribir.

Por qué al dar de baja se limpia la fecha de alta

Es contraintuitivo y por eso se comenta en el código. La regla de coherencia es: **cada usuario tiene exactamente una de las dos fechas**. Si un usuario se da de baja y se vuelve a dar de alta, la tabla de 'datos de baja' no debe seguir mostrándolo; y si se da de baja otra vez, tiene que constar la **última** baja. Limpiar la de alta al dar de baja deja el estado sin contradicciones.

Esto NO es un historial de cambios

Es la última alta y la última baja. Para tener cada cambio haría falta una tabla de auditoría aparte, que no existe y aquí no hace falta. Decirlo evita que alguien asuma que se puede preguntar '¿cuántas veces se dio de baja esta persona?' y descubra que no.

5. La caja se puede volver a dar de alta

```
@PreAuthorize("hasAuthority('ABRIR_CAJA')")
@PatchMapping("/boxes/{id}/active")
public ResponseEntity<Void> activateBox(@PathVariable Long id){
    service.activateBox(id);
    return ResponseEntity.noContent().build();
}
```

5.1 Por qué el número NO se puede reutilizar

La pregunta original era: 'cuando una caja se da de baja, ¿se puede volver a abrir una caja con ese mismo nombre?'. La respuesta es **no**, y **no hay forma de hacerlo**, y eso ya era así antes de V5:

- El **UNIQUE** está en `cash_boxes.number`.
- Dar de baja **no borra** la fila: solo pone `active = false`.
- El número sigue ocupado por esa fila, para siempre.

Reactivarla es **la misma caja**, con su mismo número y todo su historial. Y debe serlo: sus ventas apuntan a sus turnos, y los turnos a ella. Crear una caja nueva con el mismo número rompería ese encadenamiento y dejaría ventas sin caja.

Estado de la caja	Borrar	Dar de baja	Dar de alta
Nunca se abrió	Sí	Sí	No hace falta
Ya tuvo turnos	No (409)	Sí	Sí
Turno abierto	No (409)	No (409)	No (409)

6. El endpoint del historial de caja

```
@PreAuthorize("hasAuthority('VER_REPORTES')")
@GetMapping("/cash-box/{boxId}")
public CashBoxReportDTO getCashBoxReport(
    @PathVariable Long boxId,
    @RequestParam(required = false) LocalDate from,
    @RequestParam(required = false) LocalDate to,
    @RequestParam(required = false) Long userId){
    return reportsService.getCashBoxReport(boxId, from, to, userId);
}
```

6.1 Por qué se pide una CAJA y no un TURNO

Antes el selector de Reportes listaba **turnos**. Una caja abierta diez veces aparecía diez veces, todas con el mismo texto 'CAJA 1', y el usuario tenía que adivinar cuál de las diez era. La pregunta '¿qué pasó en CAJA 1?' era imposible de responder.

	Antes (por turno)	Ahora (por caja)
El dropdown muestra	Un renglón por cada apertura (10 veces 'CAJA 1')	Un renglón por caja ('CAJA 1', 'CAJA 2')
La tabla muestra	Los montos de UN turno, en vertical	Una fila por turno, en tabla
Comparar turnos	Imposible: había que memorizar el número	Una fila por turno, se comparan de un vistazo
Filtrar por usuario	No se podía: no había ni la columna	Sí, y por vendedor dentro de cada turno

6.2 La forma del DTO

```
CashBoxReportDTO // una caja
    boxId, number, description, active
    sessions -> List<CashBoxSessionDTO>
    totalSessions // SIN contar filtros

CashBoxSessionDTO // un turno
    sessionId, number, openedAt, closedAt, active
    openingAmount <- del corte CONGELADO
    cashSales / debitSales / creditSales / totalSales <- calculados
    expectedAmount, difference, differenceReason
    totalTickets, grossProfit
    sellers -> List<CashSessionSellerDTO>
    sales -> List<SaleDetailHistoryResponse>
    filtrado <- true si hay algun filtro activo

CashSessionSellerDTO // un vendedor dentro de un turno
    userId, userName, tickets, total
```

6.3 Las tres decisiones que hay que entender

El filtro se aplica en Java, no en SQL. Una caja tiene dos o tres turnos, así que el conjunto es chico y se filtra en memoria sin penalizar. A cambio, una sola pasada de código calcula las ventas, el agrupado por vendedor y la utilidad. Hacerlo en SQL serían cuatro consultas distintas y luego reconciliarlas en Java, que es justo donde se esconden los bugs.

Los montos se calculan sobre las ventas filtradas. Si el usuario filtra por Juan, el total que ve es el de Juan. El fondo inicial (openingAmount) sí viene del corte congelado, porque es dinero físico que se puso en el cajón y no es una venta.

Por qué difference llega en NULL con filtros

La diferencia del corte es un dato congelado del turno COMPLETO. Si el usuario filtra por Juan y ve 'diferencia: -50' junto a un total de \$200, el sistema estaría afirmando que faltaron \$50 en un turno del que solo se está viendo una parte. Es un descuadre que no ocurrió, y es exactamente el tipo de mentira que un corte de caja no puede decir. Por eso va en NULL y el flag filtrado le dice a la tabla que debe ocultar la columna en vez de pintar un cero.

Un turno sin ventas que cumplan el filtro no aparece. Es la decisión que se confirmó al revisar: con filtro de usuario, un turno donde no vendió ese usuario desaparece en vez de mostrar \$0. Mostrarlo con ceros sería información falsa: 'este turno no vendió nada' es muy distinto de 'este turno no aparece porque Juan no trabajó ahí'.

6.4 El agrupado por vendedor

```
// Una caja puede tener VARIOS vendedores: es lo normal, no la excepcion.
Map<Long, CashSessionSellerDTO> porUsuario = new LinkedHashMap<>();

for(SaleEntity venta : ventas){
    UserEntity usuario = venta.getUser();
    if(usuario == null){
        // Venta anterior a V5: no tiene dueño. Se agrupa aparte en vez de
        // inventar un "Desconocido" que parecería un usuario real.
        sinUsuario = sinUsuario.add(venta.getTotal());
        sinUsuarioTickets++;
    }else{
        CashSessionSellerDTO vendedor = porUsuario.computeIfAbsent(
            usuario.getId(),
            id -> new CashSessionSellerDTO(id, usuario.getName()));
        vendedor.setTickets(vendedor.getTickets() + 1);
        vendedor.setTotal(vendedor.getTotal().add(venta.getTotal()));
    }
}
```

El computeIfAbsent agrupa en una sola pasada: el mismo vendedor puede tener ventas en varios turnos y en cada uno aparece una vez. Las ventas sin dueño se agrupan aparte al final, con un nombre explícito, para que se lean como lo que son.

7. Frontend: la paginación

7.1 Las flechas de la paginación (un bug de 512 píxeles)

El botón de `<app-pagination>` ya tenía `` desde hace semanas. El problema es que los PNG miden **512×512** y no tenían ninguna regla CSS: el navegador los renderizaba a su tamaño natural y reventaba el botón por dentro.

```
.pagination-buttons button {
  display: inline-flex;      /* el <img> y el texto en una fila */
  align-items: center;
  gap: 6px;
}

.pagination-buttons button img {
  width: 14px;
  height: 14px;
  object-fit: contain;
  flex-shrink: 0;
  filter: brightness(0) invert(1); /* el PNG es negro -> blanco */
}
```

El filtro **SÍ** funciona aquí (y en otros sitios NO)

Se verificó muestreando los píxeles del archivo: `izquierda.png` es un PNG de un solo color (negro) sobre fondo transparente. Para una imagen así, `brightness(0) invert(1)` es correcto: aplasta todo a negro y lo invierte a blanco.

Ese mismo filtro aplicado a `anadir.png` **no** funciona: esa imagen tiene dos colores (una cruz verde y su contorno negro) y el filtro fusiona los dos en un bloque blanco macizo. Por eso los dos casos se tratan distinto en esta tanda.

7.2 El carrito de la compra, paginado

Cada renglón de compra tiene 6 columnas y 4 inputs. Con 30 productos el modal medía más que la pantalla y el botón 'Guardar' quedaba inalcanzable. La solución es paginar a 5 renglones y darle scroll interno.

```
lineasPage = 0;
lineasPageSize = 5;          // 5 y no 8: cada renglón es alto

/** Primer indice de `lineas` de la pagina visible. */
get lineasIndiceBase(): number {
  return this.lineasPage * this.lineasPageSize;
}

get lineasTotalPaginas(): number {
  return Math.max(1, Math.ceil(this.lineas.length / this.lineasPageSize));
}

/** Renglones de la pagina actual: los que se dibujan. */
get lineasPagina(): LineaCompra[] {
  return this.lineas.slice(this.lineasIndiceBase,
                           this.lineasIndiceBase + this.lineasPageSize);
}
```

7.3 El bug de índices que casi no se ve

Este es el detalle más importante de la paginación, y el que más bugs produce. En la plantilla:

```
<div class="linea" *ngFor="let l of lineasPagina; let i = index">
  <button (click)="quitarLinea(lineasIndiceBase + i)">    <!-- OJO -->
```

```
</div>
```

La **i** del `*ngFor` es el índice dentro de la página visible, no dentro de líneas. En la página 1 coinciden, así que el bug no se ve. En la página 2, el primer botón borraría el renglón 1 (de la página 1) en vez del 6.

El síntoma clásico de este error

Funciona en la primera página y falla en las demás. Eso hace que sea difícil de reproducir si solo se prueba con pocos productos. Es la razón por la que el comentario del template lo explica en mayúsculas: para que el próximo que lo lea no lo 'simplifique'.

Y en el template:

```
<div class="lineas-cabecera-wrap">      <!-- sticky: los titulos no se scrollan -->
  <div class="linea-cabecera">Producto | Cantidad | ...</div>
</div>

<div class="lineas-scroll">              <!-- max-height + overflow-y: auto -->
  <div class="linea" *ngFor="let l of lineasPagina; let i = index">
    ...
  </div>
```

La cabecera va **fuera** del bloque con scroll y con `position: sticky`: si se scrolleara con las filas, en el renglón 6 el usuario no sabría qué columna está leyendo.

Y un detalle de manejo de la misma clase:

```
agregarLinea() {
  this.lineas.push(this.nuevaLinea());

  // Si la compra ya crecio mas alla de la ultima pagina, se salta a esa
  // pagina para que el renglon nuevo quede a la vista. Sin esto, agregar
  // el renglon 8 con pageSize 5 lo agrega invisible (pagina 1) y parece
  // que el boton no hizo nada.
  const ultimaPagina = Math.max(0,
    Math.ceil(this.lineas.length / this.lineasPageSize) - 1);
  if(this.lineasPage < ultimaPagina) {
    this.lineasPage = ultimaPagina;
  }
}
```

8. Frontend: el modal de cajas

8.1 Los inputs que se salían de la tarjeta

Este error lo causó esta misma tanda, y es instructivo. El CSS original era:

```
.box-form input { width: 100%; }      /* MAL */
```

Y la corrección:

```
.box-form input {  
  width: 100%;  
  box-sizing: border-box;    /* indispensable */  
}
```

Por qué box-sizing no es opcional aquí

Por defecto los `<input>` usan `content-box`: el `width` se aplica al **contenido**, y el `padding` y el `borde` se suman encima. Con 10px de `padding` a cada lado y 1px de `borde`, el `input` medía 22px más que la tarjeta y se salía.

Con `border-box` el 100% ya incluye `padding` y `borde`. Es el mismo bug que provoca un `scroll horizontal fantasma` en cualquier página con un `input` al 100%.

8.2 Los cuatro botones de la tabla

La celda de acciones pasó de 1 botón a 4, y los textos tienen largos muy distintos. Con un solo nombre de clase, el CSS queda corto y el CSS no avisa. Se corrigieron dos cosas:

Botón	Cuándo aparece	Color	Por qué ese color
Editar	Siempre	Azul	Acción neutra, la de siempre
Borrar	Solo si <code>sessionsCount === 0</code>	Rojo	Es una eliminación real y definitiva
Dar de baja	Con turnos y activa	Naranja	No es borrar: el color debe distinguirlo de un borrado
Dar de alta	Si está dada de baja	Verde	Es el inverso de 'dar de baja' y tiene que leerse como tal

```
.boxes-actions {  
  display: flex;  
  gap: 6px;  
  justify-content: flex-end;  
  align-items: center;  
  flex-wrap: wrap;    /* si no caben, bajan en vez de mutilarse */  
  min-width: 0;       /* permite que los hijos se encogen */  
}
```

`min-width: 0` es la parte que se olvida: sin él, el ancho mínimo de los hijos empuja la celda, la tabla se ensancha y las columnas se descuadran. Con `flex-wrap` las acciones bajan de línea antes que partir un texto a la mitad.

8.3 El icono del botón Crear caja

añadir.png es una cruz **verde**. Dos decisiones que van juntas:

- **Sin filtro de color:** herechar la cruz la convertiría en un bloque blanco (el problema descrito en §7.1).
- **Fondo oscuro detrás del icono:** cruz verde sobre botón verde es invisible, así que el icono lleva un círculo oscuro que lo separa.

```
.btn-confirm-open img {  
  width: 16px; height: 16px;  
  object-fit: contain;  
  background: #0f172a;      /* separa la cruz verde del boton verde */  
  border-radius: 50%;  
  padding: 2px;  
}
```

9. Frontend: el historial de caja en Reportes

9.1 Dónde viven los dos filtros

Fue una aclaración importante del usuario y define toda la estructura:

Filtro	Dónde vive	Por qué
Caja	En la barra de filtros general de arriba	Es un filtro de la consulta, como las fechas
Usuario	Dentro de la sección de caja	Solo tiene sentido respecto a una caja elegida

9.2 El filtro de usuario se arma con los que venden ahí

```
get boxReportUsers(): { userId: number | null; userName: string }[] {
  if (!this.cashBoxReport) return [];

  const porId = new Map<number | null, { userId, userName }>();

  for (const sesion of this.cashBoxReport.sessions) {
    for (const v of sesion.sellers) {
      porId.set(v.userId, { userId: v.userId, userName: v.userName });
    }
  }

  // Null (sin usuario) al final: es el caso excepcional, no el principal.
  return [...porId.values()].sort((a, b) => {
    if (a.userId === null) return 1;
    if (b.userId === null) return -1;
    return a.userName.localeCompare(b.userName);
  });
}
```

Por qué NO es la lista de usuarios del sistema

En un local con 8 empleados, ofrecer los 8 cuando solo 2 trabajan en esa caja produce 6 opciones que devuelven una tabla vacía. Y el usuario no puede distinguir 'sin resultados' de 'este usuario no vendió aquí'. La lista sale de los sellers que ya vienen en cada sesión, así que no cuesta nada.

Una caja puede tener **varios** vendedores, y eso es lo normal: la lista los muestra todos. El set en vez de add es porque el mismo vendedor aparece en varios turnos y en el dropdown debe salir una sola vez.

9.3 Por qué el filtro va al backend

Los filtros se envían como parámetros, no se aplican en el navegador. Si se filtrara en el frontend habría que traer todas las ventas de todos los turnos para descartar la mayoría en el cliente, y los totales de la tabla no coincidirían con los del backend. Un filtro que no coincide con los números que muestra es peor que no tenerlo.

```
getCashBoxReport(boxId: number, filters: {...} = {}) {
  return this.http.get<CashBoxReportDTO>(`${this.api}/cash-box/${boxId}`, {
    params: buildParams({
      from: filters.from ?? undefined,
      to: filters.to ?? undefined,
      userId: filters.userId ?? undefined,
    }),
  });
}
```

El detalle de null en buildParams

La función que arma el query string filtraba undefined y '', pero **no** null. Un <select> sin opción elegida vale

null, y `String(null)` es la cadena "null", que el backend no puede convertir a Long y respondería 400. Por eso el filtro agrega `!== null`.

9.4 Un turno a la vez

El detalle de las ventas se despliega con un clic en la fila, como en el detalle de renglones de Compras. Solo uno a la vez, y eso obliga a que el estado sea un id y no una lista:

```
sessionAbiertaId: number | null = null;

toggleSesion(sessionId: number) {
  this.sessionAbiertaId =
    this.sessionAbiertaId === sessionId ? null : sessionId;
}
```

Con dos turnos abiertos a la vez la tabla duplicaría su alto y se perdería de vista de qué turno es cada bloque.

9.5 `?? 0` en la plantilla

```
<td class="money" *ngIf="!s.filtrado"
  [class.dif-negativa]="(s.difference ?? 0) < 0">
  {{ (s.difference ?? 0) > 0 ? '+' : '' }}{{ (s.difference ?? 0) | number:'1.2-2' }}
</td>
```

Hace falta porque `difference` es `number | null` (null cuando hay filtro) y comparar `null < 0` es un error de tipos que Angular reporta como 'Object is possibly null'. Como la celda solo se pinta cuando NO hay filtro, el 0 nunca llega a verse.

10. Frontend: buscar en usuarios dados de baja

10.1 Qué se agregó

La tabla de /usuarios-desactivados no tenía nada para buscar. Se le agregaron tres filtros y dos columnas, para responder una pregunta que antes no tenía respuesta: '¿cuándo se fue esta persona?'.

Filtro	Sobre qué dato	Nota
Texto	NOMBRE y CORREO	Con las dos columnas: quien busca un registro muchas veces recuerda el correo
Desde	Fecha de baja	Compara solo la parte yyyy-MM-dd
Hasta	Fecha de baja	

10.2 El detalle de las fechas (y por qué no se usan objetos Date)

```
if (this.fromDate || this.toDate) {  
  if (!u.deactivatedAt) return false; // sin fecha: no entra al rango  
  
  const dia = u.deactivatedAt.substring(0, 10); // "2026-10-01"  
  if (this.fromDate && dia < this.fromDate) return false;  
  if (this.toDate && dia > this.toDate) return false;  
}
```

Por qué comparar STRINGS y no objetos Date

El input type="date" entrega "2026-10-01". El orden lexicográfico de ese formato **es** el orden cronológico, así que comparar strings funciona y evita cualquier conversión.

Si se usara `new Date( . . . )` la zona horaria haría que un registro del 1/10 se evaluara como 30/09 en un huso como el de México (UTC-6). Ese bug desplaza un día los bordes del rango y es de los que cuesta encontrar.

10.3 El aviso de los usuarios sin fecha

Los usuarios dados de baja **antes** de V5 no tienen `deactivatedAt`: la columna se agregó después. Con un filtro de fechas quedan fuera, y sin aviso el usuario pensaría que desaparecieron. Por eso el contador:

```
get sinFecha(): number {  
  return this.users.filter(u => !u.deactivatedAt).length;  
}
```

El contador se calcula sobre la lista **completa**, no sobre la filtrada. Si se calculara sobre la filtrada daría siempre 0 (los que no tienen fecha ya quedaron excluidos por el filtro) y el aviso nunca aparecería. Es un detalle fácil de escribir mal.

10.4 Los dos estados vacíos distintos

```
<tr *ngIf="!loading && users.length === 0">  
  No hay usuarios dados de baja <-- nunca hubo usuarios de baja  
<tr *ngIf="!loading && users.length > 0 && filteredUsers.length === 0">  
  Ningun usuario dado de baja coincide con los filtros
```

Son mensajes distintos porque son hechos distintos. 'No hay usuarios dados de baja' es un dato del sistema; 'ninguno coincide con los filtros' es un dato sobre lo que el usuario acaba de escribir, y lo que tiene que arreglar es el filtro, no el sistema.

11. Frontend: alinear las columnas del historial

El bug era sutil: los datos de las columnas de dinero iban a la derecha (clase `.money`) pero sus títulos iban a la izquierda. El resultado era una tabla que se veía corrida aunque los números estuvieran perfectamente alineados entre sí.

```
<th class="col-id">#</th>
<th class="col-fecha">Fecha</th>
<th>Método de pago</th>
<th class="money">Efectivo</th>      <!-- <- antes sin clase -->
<th class="money">Cambio</th>
<th class="money">Total</th>

<td class="col-id">{{ sale.id }}</td>
<td class="col-fecha">{{ sale.saleDate | date:'dd/MM/yyyy HH:mm' }}</td>
<td class="money">...</td>
```

La regla que deja todo resuelto

Si una columna tiene los datos alineados a la derecha, su título también. La clase de alineación va en el `th` y en el `td` por igual, y las dos columnas quedan sobre sus datos sin tocar nada más.

Dos detalles más del mismo arreglo: `col-id` centra el número de ticket (es un identificador, no una cantidad, y centrado se escanea mejor en vertical), y `col-fecha` lleva `white-space: nowrap` porque `'04/10/2026 14:30'` partido en dos líneas hace la fila el doble de alta.

12. Cómo se verificó

Qué	Cómo	Resultado
Backend completo	mvnw clean test	258/258
Frontend completo	ng build	OK
Alineación de columnas	Cada th lleva la misma clase de alineación que su td	Corregido
Índices del carrito	quitarLinea(lineasIndiceBase + i)	Corregido antes de que se notara

12.1 Qué falta probar a mano

- Vender algo y ver que el reporte de la caja muestra el nombre del usuario en el detalle del turno.
- Dar de baja una caja, volver a darla de alta y comprobar que conserva sus turnos y que el número sigue ocupado.
- Dar de alta una caja y filtrar en Reportes por un usuario que no vendió en ella: la tabla debe salir vacía, no con ceros.
- En la tabla de sesiones, poner un filtro de fecha y ver que la columna Diferencia muestra '—' en vez de un número.
- En usuarios dados de baja, filtrar por un rango de fechas y ver el aviso de los usuarios sin fecha.

Sobre los tests del frontend

ng test sigue diciendo 'No test files found'. No es un defecto del código: el path del repositorio tiene paréntesis y rompen el glob de vitest. Está documentado en el AGENTS.md del frontend desde 2026-09-30. Para correrlos hay que mover el proyecto a una ruta sin paréntesis.

13. Lo que resume esta tanda

Motivo	Qué se hizo
'No sé quién vendió esto'	La venta guarda su usuario, tomado del token
'No sé cuándo se fue esta persona'	Dos fechas en el usuario, coherentes con su estado
'Esta caja se dio de baja y ya no vuelve'	Re-alta: la misma caja, mismo número, mismo historial
'En Reportes veo CAJA 1 diez veces'	El selector lista cajas; la tabla lista sus turnos
'No encuentro a un usuario que se fue'	Filtros por nombre, correo y rango de fechas
'La tabla se ve corrida'	Cada título con la misma alineación que sus datos
'El modal no cabe en la pantalla'	El carrito se pagina y tiene scroll propio

Casi todo lo de esta tanda sale de la misma pregunta mal hecha: **¿qué pasa cuando algo se repite?** Una caja se abre cada día, una persona entra y sale del equipo varias veces, y la respuesta buena no es acotar el caso sino **cambiar el modelo** para que lo repetido no sea una excepción. La caja dejó de ser el turno (V4), el usuario dejó de ser un booleano (V5), y la venta dejó de ser un número suelto (V5).